FH University of Applied Sciences TECHNIKUM WIEN

BACHELOR PAPER

Thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science in Engineering at the University of Applied Sciences Technikum Wien - Degree Program BIF

Development of a root-state and child-state abstraction algorithm for the game 2048.

By: Stefan Werner
Student Number: 2310257227

Supervisor: Degree Marius Führer, Degree

Vienna, May 05, 2026

Declaration of Authenticity

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz/ Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and according to the rules currently applicable at the UAS Technikum Wien and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.”

Place, Date	Digital Signature

Kurzfassung

Diese Arbeit untersucht eine Methode zur Reduzierung des Speicherbedarfs von Q-Tabellen am Beispiel des Spiels 2048. Beim Standard Q-Learning wird jeder beobachteter Zustand zusammen mit seinem zugehörigen Aktionswerten gespeichert. Dadurch steigt der Speicherbedarf mit zunehmender Anzahl an Zuständen stark an. Der vorgeschlagene Ansatz teilt die ursprüngliche Q-Tabelle in zwei Strukturen auf. Eine Eltern Tabelle und eine Kind Tabelle. Die Eltern Tabelle speichert vollständige Zustände und Aktions paare. Die Kind Tabelle speichert einen Verweis auf einen Eltern Zustand, eine Operationssequenz und die ursprüngliche beste Aktion. Die Abstraktion basiert auf der Annahme, dass strukturell ähnliche Zustände im Spiel 2048 durch Matrixoperationen wie Rotation, Spiegelung und Verdopplung dargestellt werden können. Während der Rekonstruktion werden die gespeicherten Operationen auf den Eltern Zustand angewendet, um den jeweiligen Kind Zustand wiederherzustellen. Die rekonstruierte Q-Tabelle wird anschließend mit der ursprünglichen Q-Tabelle verglichen. Die Ergebnisse zeigen, dass die Wirksamkeit der Abstraktion stark an der Größe der Umgebung also dem Spiel und der Tiefe des Suchalgorithmus abhängt. Die stärkste Speicherreduktion wurde in der 3 bei 3 Umgebung erreicht. Hier wurde der Speicherbedarf um bis zu 56 Prozent reduziert und die Policy Übereinstimmung war dennoch über 99,6 Prozent. Insgesamt zeigen die Ergebnisse, dass die Eltern Kind Abstraktion den Speicherbedarf reduzieren und gleichzeitig die gelernt Entscheidungsstrategie in kleinen und mittleren Umgebungen weitgehend erhalten kann. Für größere Umgebungen sind jedoch weitere Optimierungen des Trennungsalgorithmus, der Eltern Auswahl und der Operationsmenge erforderlich.

Schlagwörter: Reinforcement Learning, 2048, Q-Table, Abstraktion, Machine Learning, State Abstraktion, Matrix

Abstract

This work investigates a method for reducing the memory requirements of Q-tables using the game 2048 as an example. In standard Q-learning, each observed state is stored separately together with its corresponding action values. This leads to increased memory requirements as the number of states increases. The proposed approach addresses this problem by dividing the original Q-table into two structures, a parent Q-table and a child table. The parent saved complete state-action pairs without abstraction, and the child stores a reference to the parent as well as an abstraction operation list. This list can later be used to rebuild each state from its parent. The abstractions are based on the assumption that structurally similar states in the game 2048 can be represented through matrix operations. During the reconstruction step the stored operations are applied to the parent to rebuild each child. The resulting Q-table is compared to the original Q-table. The results show that the effectiveness of the abstraction depends strongly on the size of the environment and the used depth of the search algorithm. The strongest reduction in storage space was achieved in a 3 by 3 environment, where the parent child representation reduced the storage requirements by up to 56 percent while maintaining a policy agreement above 99,6 percent. Overall, the results show that parent child Q-table abstraction can reduce storage requirements while preserving the learned policy in smaller and medium sized environments. However further research and optimization on the splitting algorithm and parent selection as well as operations sets is required for it to become practical in larger environments.

Keywords: Reinforcement Learning, 2048, Q-Table, Abstraction, Machine Learning, State Abstraction, Matrix

Acknowledgements

I would like to thank my parents and sister for giving me the opportunity to be able to get this far. It wasn’t easy and their constant support and patience helped me in the darkest times.
I would also like to thank my supervisor for the guidance and support during the development of this thesis. Finally, I would like to thank everyone who helped by reviewing parts of this work and giving helpful criticism.

Table of Contents

1 Introduction

Reinforcement Learning (RL) has become one of the most widely used approaches for solving decision making problems [1]. Especially for games or environments where modelling the entire Environment is difficult or impossible such as self-driving cars [2]. One of the most fundamental methods or learning structures for RL is called Q-learning [3]. The agent learns connections between states and actions by storing the best move in a Q-Table. Q-learning works great for small environments where the state space is minor but suffers from a major limitation, the size of the Q-table grows increasingly with the number of different states [4]. This can lead to significant memory requirements and inefficiencies in storing this large number of states.

This thesis focuses on the game 2048 as an example of increasing state space size due to its exponential state growth [5]. Even small versions of the game with a smaller grid can lead to a large number of possible states. To address this issue, this thesis introduces a abstraction approach based on a parent child concept [6]. The core idea is the reduction of saved space by not saving every state but only saving root or in this thesis also called parent states and their corresponding child states. Child states are determined though matrix operations such as rotation doubling and edge detection. [7] [8]

1.1 Motivation

The Q-table is essentially the brain of the agent. It stores the connection between state and action, creating a state-action pair and directly determines the behavior of the agent during the decision-making process. As previously stated in small environments, this approach is efficient and easy to implement. But in more complex environments or non-deterministic environments this can easily lead to millions of state-action pairs with very similar states and actions. This becomes particularly problematic with scaling as a larger grid in a game like 2048 increases the state space dramatically [4]. A 4 by 4 2048 Q-table can easily exceed 10 million states (~ 1 GB). This approach could reduce the number of saved states by half [9] [10].
Many of these states are essentially the same just rotated or doubled and lead to wasted space which could be used to save more important states that are underrepresented in the Q-Table [7] [9] [10].

1.2 Problem statement

The problem addressed in this thesis is the inefficient storage of Q-table entries in environments with large state spaces.
Traditionally, in Q-learning every state-action pair is stored separately regardless of the similarities between the pairs. This leads to redundant information being stored multiple times, increasing memory usage without adding new knowledge.

The idea is to design a method that reduces the number of stored states, avoids loss of information and allows reconstruction of the original Q-table when needed.
In particular, the problem can be formulated as follows:
How can a Q-table be compressed by identifying relationships between states, such that redundant states are represented through transformations of a smaller set of base states, without significantly degrading the learned policy?

1.3 Research Questions

1. How can similarities between states in the game 2048 be identified and represented?
2. Can a Q-table be compressed by representing states as children of other states through matrix transformation?
3. How does this abstraction approach effect the performance of the agent?
4. What is the trade of or overhead of this approach?
5. How does transformation depth affect compression ratio and policy accuracy?
6. What are the limitations of this approach and what can not be done?

2 Methodology

This chapter describes the approach used to develop the proposed root-state and child-state Q-table abstraction algorithm. The purpose of this thesis is the development and evaluation of a form of Q-table abstraction by representing states with structural similarities as a parent-child link where the child inherits the action-values of the parent.
The proposed abstraction method relies on measurable factors such as compression ratio, reconstruction accuracy, policy preservation, runtime and memory as well as CPU overhead. Therefore a theoretical description of the method was deemed useless if not accompanied by a practical example. The method was designed, implemented, tested and evaluated in multiple forms and iterations. A background of the game 2048 and reinforcement learning is necessary to understand the solution which can be found in appendix F.

2.1 Ideal solution

The ideal solution is designed independently of implementation constraints such as language, storage format, disk space, RAM usage and time constraints. In the ideal model the original Q-table is not stored as a singular file of state-action pairs but is instead stored as a combination of one or more smaller files where structurally related states are grouped together. This grouping would be called the list of parent states.

A parent state, in this thesis also called root state, is a state which has a specific form or layout that it shares with other states. A parent state is therefore a representation of many other states

that are similar. Other states that can be derived from such a state are called child states. A child state can only have one parent state but parent states can have an indefinite amount of children. A child state can be derived from its parent state with the help of a sequence of transformations or in this thesis called operations that are stored with the child state. The sequence of operations must be apply able to the state and to the paired actions of the parent in order for the child to be able to be derived. If a state-action pair can not be perfectly restored to its original pairing a loss of information will be the consequences.
The child state does not store the complete set of Q-values. It only stores the following:

1. A reference to the parent state, this can be done via a key or index
2. The sequence of operations required to derive the child state, a string of characters or a list of instructions
3. The original best action of the child state, used to reduce the loss of information

The ideal goal is for the reconstructed Q-table to be equal to the original Q-table. The two tables are equal if the preserved decision behavior of the agent remains the same. This should be achieved while requiring less storage space. The ideal would be for the Q-values to be numerically the same but as this cannot be achieved the main objective is more that the resulting policy remains as similar as possible.
The game 2048 was chosen as it contains many state with similar spatial structures. Rotation and mirroring operations can preserve structural meaning of the board if the action-values are transformed accordingly. A operations such as doubling the values of each tile are treated as approximate similarities since they preserve the spatial pattern. Doubling the values may change the reward scale and the game phase which needs to be factored into the final assessment.

The ideal solution is based on the assumption that every state in the game 2048 has a state that has at least one similarity with another state. In the best case states with structural equivalent states can be grouped together as a single parent state.
A state in the game 2048 is a four by four matrix where every tile can have a value dividable by two or be the value zero.

Figure 1 Example of a state in the game 2048

The ideal abstraction mapping defines that each state s is a child of a representative parent state p.

Let S be the set of all states in the original Q-table and $P \subseteq S$ the set of parent state such that:

$$S \to P \tag{1}$$

Which assigns each state $s \in S$ to a representative parent state $p \in P$. A child state c can be represented by a parent state p if there exists a sequence of transformation operations O such that:

$$c = O(p) \tag{2}$$

In this model the parent states would store the complete state-action pair. The child stores the reference to the parent and the operations. The ideal abstraction is a lossless abstraction in which the reconstructed Q-table produces the same policy with no loss of information such that:

$$argmax_a\, Q_{original}(s,a) = argmax_a\, Q_{reconstructed}(s,a) \tag{3}$$

In the ideal solution parent child classification would happen during training. Each newly observed state would immediately be checked against already known parent states. If the state could be derived from an existing parent through an operation sequence it would be stored as a child. Otherwise it would be classified and saved as a new parent. More on the ideal solution can be found in appendix B.

2.2 Requirements

As previously mentioned, the thesis relies on measurable factors therefore the functional requirements were derived from the need of a functioning and reliable environment and agent cycle.

2.3 Development process

The development process followed an iterative prototyping approach. First the environment was implemented. In order to ease testing and for convenience the game 2048 was created in such a way that the world can be created with different sizes. The rest of the implementation was created in several phases. Each phase produces an intermediate result that was tested and used as the input for the next phase. The first phase was the creation of the Q-learning agent. The agent choses an action and updates its Q-table based on the rewards it receives from the environment. The second phase focused on the design and implementation of transformation operations. These operations where required for the agent to determine if two states are child and parent. The child and parent tables are saved as separate tables. The last phase was the creation of the reconstruction step. The reconstruction step is done at the beginning of the training or play function of the agent. It combines the parent and child tables into the original Q-table. This reconstructed table is then used to play the game. A explanation for all phases of the development process can be found in appendix C.

2.4 Testing and evaluation method

The testing class was designed to determine whether the proposed parent-child abstraction satisfies the requirements of this thesis. The evaluation focuses on the main requirements of this paper. It investigates whether the abstraction idea of parent and child reduces the storage requirements of the original Q-table. Additionally, it tests the loss of information during the splitting and reconstruction step of this thesis. The final evaluation was done by generating Q-tables for different versions of the 2048 environment. A two by two environment which was used for early correctness testing. The smaller state space made it easier to manually inspect states, operations and reconstruction results. The three by three environment was used to test the behavior of the algorithm on a larger and more complex state space while still keeping the experiment computationally manageable. The four by four environment represents the original version of the game and was used to evaluate the practical applicability of the approach.
The testing process consists of several evaluation metrics.

$M_{reduction}$	$Storage\ reduction$
$S'_{abstraction}$	$child\ abstraction\ ratio$
P_C	$child\ per\ parent\ ratio$

$\pi_{agreement}$	$Policy\ agreement$
Q_{error}	$Q-value\ error$
$T_{overhead}$	$Runtime\ overhead$
$M_{overhead}$	$Memory\ oberhead$
$Q_{average}$	$best\ average\ Q-table$

The evaluation metrics are discussed in detail in appendix D.

2.5 Tool selection

The tools selected for the implementation were based on the requirements. Python was chosen as the prototyping language. NumPy was used for representing transforming game states and the results were shown in a CSV format. A detailed explanation for every tool used can be found in appendix E.

3 Solution

This chapter describes the program and the implemented solution to the proposed root-state and child-state Q-table abstraction algorithm. It includes the required knowledge needed to understand the game 2048 and a simplified explanation of Q-learning. Both define the structure of the state space and the Q-table used by the prototype.

3.1 Notation

3.1.1 Reinforcement Learning Notation

RL often uses a standardized notation. Here, s denotes a state, a an action, and r the reward received. The policy π describes the agent's decision rule. The following notation is often used in Reinforcement Learning [1]. A continuation of the notation can be found in appendix A.

s	$state$
a	$action$
t	$timestep/repetition$
r	$reward$
e	$episodes$
γ	$discount\ Factor$
ϵ	$exploration\ rate$
$1-\epsilon$	$exploitation\ rate\ (probability\ of\ using\ the\ q-table)$
α	$learning\ rate$

S $set\ of\ all\ states$
A $set\ of\ all\ actions$

[1]

3.1.2 Abstraction Algorithm Notation

The following notation is specifically used for this bachelor's thesis and describes the relationship between root and child states.

p $root/parent\ state$
c $child\ state$
o $operation$

P $set\ of\ all\ root/parent\ states$
C $set\ of\ all\ child\ states$
$\mathcal{O}$ $set\ of\ all\ operations$
O $composition\ of\ operations$

M $memory\ usage\ of\ non\ abstracted\ Q-table$
M' $memory\ usage\ of\ abstracted\ Q-table$

T $time\ usage\ of\ non\ abstracted\ Q-table$
T' $time\ usage\ of\ abstracted\ Q-table$

3.2 Parent-Child Abstraction Concept

This thesis introduces a Q-table abstraction based on a parent-child representation of states. In a normal Q-learning setup for the game 2048, the Q-table grows exponentially. This leads to high memory consumption and inefficient storage. The core idea of this work is to reduce the number of stored states by identifying structural similarities between them. Instead of saving every state individually states are grouped into equivalence classes based on matrix transformations. Within each group a parent state represents the best action for all other states. This parent state is stored explicitly while all other states are saved with a key to the parent, the matrix operation and the previously best direction for the state.

3.2.1 Mathematical definition

Defining this behavior mathematically. The set of parent states P is defined as a subset of the set of all states S:

$$P \subseteq S \tag{26}$$

A state s is classified as a parent state p if it cannot be generated from any other distinct state using a sequence of allowed operations and if p is an element of all the states S.

$$p \in P \iff \nexists s \in S, s \neq p, \exists O: p = O(s) \tag{27}$$

The set of child states C is defined as a subset of the set of all states S.

$$C \subseteq S \tag{28}$$

Every child state c in the set of all child states C is classified as a child state if and only if there exists exactly one parent state p in the set of all parent states P and a sequence of transformation operations that generate a child state c from the parent state p, where $c \neq p$.

$$c \in C \Leftrightarrow \exists! p \in P, \exists O: c = O(p), c \neq p \tag{29}$$

The way p is chosen is by sorting the Q-table by number of zeros in it and choosing the first matching parent in the sorted order.

$$\begin{gathered} z(s) = number\ of\ zero\ entries\ in\ s \\ s_i \prec s_j \iff z(s_i) > z(s_j) \end{gathered} \tag{30}$$

$$f(s_i) = \begin{cases} p_k, & where\ p_k = \min\{p \in P | \exists O: s_i = O(p)\} \\ s_i, & otherwise \end{cases} \tag{31}$$

3.2.2 Parent-Child representation

Using the definitions above the original Q-table is divided into two components. A Q-table filled with parent state-action pairs:

$$(index, state, actionUP, actionDOWN, actionLEFT, actionRIGHT)$$

The index is used for the child to link to the parent. The state is a flattened matrix. The actions represent a float value of each direction on the grid chosen by the agent.

$$(1, "0,0,2,0,2,4,2,4,8", 23.34, 13.3, 10.534, 32.5758)$$

The second part of the divided Q-table is the child Q-table filled with the index of the parent and operations applied to the parent to get to the original child.

$$(parentIndex, operations, direction)$$

The index is a key that points to the original parent and the operations are a string of operations applied to the child to get to the parent. The direction property insures the agent doesn't lose context of the state.

$$(12, \text{"rdde"}, 1)$$

3.3 Transformation Operations

The parent child abstraction algorithm uses transformation operations to determine whether a state can be represented through an already known parent state. In the prototype two states are considered equivalent if one can be transformed into the other using a sequence of predefined matrix operations. Formally a state s can be a child state c of a parent state p if there exists at least one sequence of operations such as:

$$s_1 \sim s_2 \Leftrightarrow \exists O: s_1 = O(s_2) \tag{32}$$

In the context of the parent child abstraction a child state c can be represented by a parent state p if the child can be reconstructed by applying the stored operation sequence in its Q-table.

$$c = O(p) \tag{33}$$

A stored operation sequence of a child c can look like:

$$r$$
$$ddr$$
$$mrdre$$

The sequences are defined as the following:

$$O = (o_1, o_2, \dots, o_k) \tag{34}$$

Where o_i is an element of the set of all possible operations:

$$o_i \in \mathcal{O} \tag{35}$$

A sequence is applied to a parent in an additive loop to generate a child state:

$$c = O(p) = r(d(r(p))) \tag{36}$$

Or mathematically:

$$c = O(p) = o_k(\dots o_1(p) \dots) \tag{37}$$

In the implementation transformation operations are not only applied to the state but also to the corresponding Q-values of the parent in order to return the child's original actions. This is necessary because the Q-values are linked to the direction up, down, left and right in relation to the orientation of the state. A rotated state needs therefore also rotated action-values. The child tale saves the parents index, the sequence of operations outlined above and the original best direction of the child's state. The best direction is used as an additional safeguard if the operations applied to the action change the Q-values of the child. If the original action are not the same the agent can minimally adjust the reconstructed action values. The implemented prototype uses three transformation types, rotation, mirroring and scaling or doubling. Rotation and mirroring change the orientation of the board and therefore require corresponding transformations of the action values. Doubling scales the tile values but does not change the orientation and therefore the actions remain unchanged. Detailed examples of each operation are provided in Appendix B.

3.3.1 Splitting

In this thesis the original Q-table is split into two tables a child table and a parent table. The following equations describe this mathematically:
A parent state-action pair consists of the parent state and the actions for the given state.

$$p_{sa} = (p, q_p) \tag{38}$$

With:

$$q_p = (Q_p^{up}, Q_p^{down}, Q_p^{left}, Q_p^{right}) \tag{39}$$

q_p Q-values for parent state-action pair
Q_p Q-value for parent

A child table consists of the index of the parent state-action pair, the operational sequence and the best saved direction:

$$c = (p, O, d_c) \tag{40}$$

- $p \in P$ the parent state for the child
- O Operational sequence
- $d_c \in A$ the saved best direction for the child state

Example:

$$p_{sa} = (1; 0,2,2,0; 25.34,12.23,29.123,26.296)$$
$$c_{sa} = (0,4,4,0; 25.34,21.56,25.23,22.896; 0)$$

$$c = O(p) + direction\_c \tag{41}$$

$$c = (1; d; 0)$$

3.3.2 Reconstruction

The reconstructed state of the child is generated with the operational sequence saved in the child plus the parent state.

$$c = O(p) \tag{42}$$

Additionally the transformation must also be applied to the actions of the parent:

$$T_O: \mathbb{R}^4 \rightarrow \mathbb{R}^4 \tag{43}$$

$$q'_c = T_O(q_p) \tag{44}$$

q'_c the reconstructed actions from the parent state-action pair to the child

If the best direction of the child and the best direction of the parent is not the same the actions will be transformed to fit the best direction.

$$\hat{d}_c = arg \max_{a \in A} q'_c(a) \tag{45}$$

If the best direction of the child and the best direction of the parent is the same the actions of the parent are applied to the child.

$$\hat{d}_c = d_c \tag{46}$$

$$q_c = q'_c \tag{47}$$

Example:

$$s = O(c) \tag{48}$$

$$s = (0,4,4,0;\ 0;\ 25.34,12.23,29.123,26.296;\ 0)$$

If $bestDirection \neq direction\_s$:

$$apply\ best\ direction\ (s)$$
$$s = (0,4,4,0;\ 0;\ 25.34,12.23,25.33,25.34;\ 0)$$

3.4 Environment

The environment in this thesis is the game 2048. The environment consists of 3 versions. A 2 by 2 grid version a 3 by 3 version and the normal 4 by 4 version. As previously mentioned the environments were chosen to test different results and conditions. The total amount of states found in the 2 by 2 grid is 476. This was found after 10 hours of continuous running of the program. The total state space of the 3 by 3 version and the 4 by 4 version has not yet been reached and takes up to much storage space for this thesis. The current state space is 295.330 for the 3 by 3 version and 2.554.721 for the 4 by 4 version. More on the environment in the Appendix G.

3.5 Agent

The agent implements a value-based reinforcement learning approach. It uses a Q-table and learns a mapping from state to action values like a normal value-based reinforcement learning model. The difference is that the state action pairs are not saved as a singular Q-table but saved as a parent and child state Q-table. The learning process itself is still the classical idea of storing and updating values for states on a singular Q-table.

In each step, the agent observes the current grid finds a set of valid actions, selects one of these actions according to its exploration strategy, and updates its stored action values based on the reward returned by the environment. After a certain amount of states determined by a user chosen batch size the singular Q-table is split into child and parent state. To determine if a child state is a child of a parent state the agent looks through every state and compares them with each other. If two states are equivalent under the allowed matrix operations, one state can be stored explicitly as a parent while the other is represented as a child together with the transformation path and the saved best direction.

The agent therefore has two roles. It acts as the learning component of the reinforcement learning loop and manages the abstraction pipeline. It provides the functionality to load and save the child and parent Q-tables divide the Q-table used for the learning path of the agent

and reconstructs the children and parent states to a singular reconstructed Q-table for continued training. More information can be found in the Appendix H.

3.6 Splitting algorithm

The PreCalculator is the python script that includes the previously mentioned operations. It is called in order to split an existing Q-table into child and parent. The PreCalculator is responsible for exploring the transformation space of a given parent state. It is given a parent state and determines all possible reachable child states. It then saves all operations and child states in a list for later comparison. If a state matches with one of these calculated child states the operations are saved to that child state and the child is recognized as a child of the parent. If a given state is not a possible child of a parent state the state becomes a parent itself and the PreCalculator is called again. The new calculated states are appended to the existing list of reachable states.

This implementation is made with a breadth-first search strategy. The search begins with the original parent state at depth 0 and an empty operation path. After each visited node the PreCalculator applies the available operations and inserts the resulting states into the search queue for the next state to check against. The process continues until the maximum depth is reached.

Each generated result contains the following:

- The transformed state
- The transformed action values
- The sequence of operations used to reach that state

3.7 Reconstruction algorithm

To evaluate the abstraction the parent and child tables are merged back into one reconstructed Q-table. This table is then compared to the original Q-table. The reconstruction begins with the table of parent states. It is loaded into memory as the parent states already contain full state-action information. Then the program runs through each child state and applies the operations in order to return the children to complete states. For each child the parent state is loaded and the operations are applied. Then the actions of the parent are applied to the child and the action transformation takes place. If the best direction stored in the child does not match with the actions of the parents the actions will be minimally adjusted until the best direction of the child matches. This is done via small value offsets ensuring that the original action structure is changed as little as possible.

3.8 Experimental results

In this chapter the results of the parent-child abstraction will be presented. As mentioned in chapter 2.4 the evaluation metrics are a result of time overhead, memory overhead and difference between original Q-table and reconstructed Q-table. The results were gotten from the three different grid sizes of the environment. For each grid size the same evaluation pipeline was applied. First the agent was trained and a normal Q-table was created. Then the parent child abstraction idea was implemented. The Q-table was split via the PreCalculator algorithm and was saved as a parent table and a child table. For comparison the reconstructed Q-table was generated at the end from the parent and the child tables. The following tables and graphs compare the abstraction algorithm compared to the not abstracted original Q-table. A detailed explanation of every metric and the reason for its use can be found in appendix D.

For the following metrics the exploration of the data would have taken up to much of a word count and the detailed explanation as well as the results one can surmise from it are therefore in the appendix I.

3.8.1 Storage Reduction

The storage reduction $M_{reduction}$ compares the size of the original Q-table with the combined size of the parent and child tables M'. The reconstructed Q-table is not included in the M'. As previously defined in chapter 2.4 the storage reduction is calculated as:

$$M_{reduction} = 1 - M'/M \tag{5}$$

Where M is the storage size of the original Q-table and M' is the combined storage size of the parent and child tables.

$$M' = M_P + M_C \tag{4}$$

Here, M_P is the storage size of the parent table and M_C is the storage size of the child table. A value of 0 means that no storage was saved. A positiv value means that the parent child abstraction requires less storage than the original Q-table and a negative value means that the abstraction requires more storage than the original representation. Table 1 shows the measured storage size for the different grid sizes and the transformation depths.

Table 1 Storage reduction percentage

Storage Reduction %					
Grid Size	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	- 4%	11%	18%	21%	25%
3x3	28%	55%	55%	55%	56%

4x4	3%	6%	5%	5%	5%

The 2 by 2 environment show the smallest absolute storage sizes. At PreCalculator Depth one the storage reduction is negative with -4 percent. This means that the parent child abstraction is larger than the original Q-table. This makes sense as the original Q-table is very small and a depth of 1 makes to little impact for children to be found. But the abstraction still introduces additional overhead through the child table and the separate file structure. An abstraction with no child abstraction still creates a child table file, an empty one but still a file. With increasing transformation depth the storage reduction improves. The depth steps are increased by 2 each step. At the next depth 3 the abstraction saves 11 percent of the storage space. At depth 10 and at grid size 2 the reduction reaches its max with 25 percent. This shows that the abstraction becomes useful for the two by two environment only when enough child states are detected to compensate for the additional time overhead.
Larger grid sizes and their impact on the data are discussed in appendix I.

The following figure visualizes the storage usage for the 2 by 2 approach in KB. The normal Q-table remains at a constant 28 KB while the parent table decreases from 28 KB at its max in depth 1 to 18 KB at depth 10. The child table grows from 1 KB to 3 KB. This shows that the combined storage size of parent and child M' still decrease from 29 KB to 21 KB saving 7 KB of space compared to the original not abstracted Q-table.

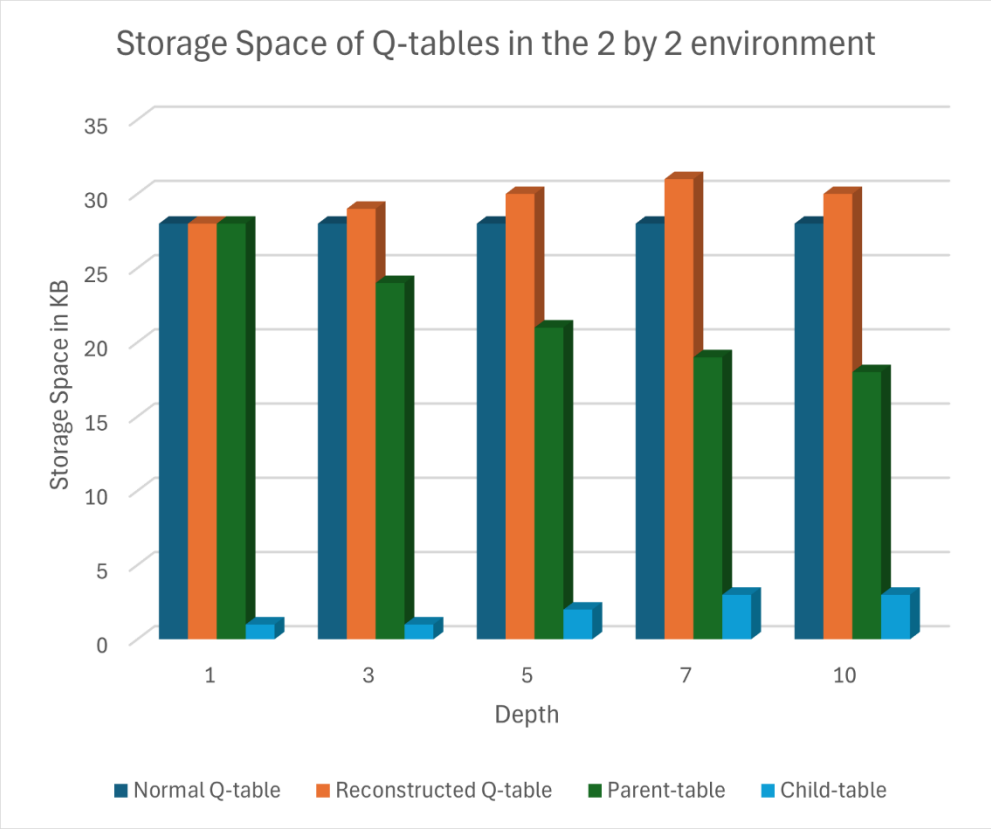

Figure 2 Storage Space of Q-table in 2 by 2 environment

Figure 13 represents the storage usage for the 3 by 3 environment. It shows the decreased storage space required due to the abstraction in KB.

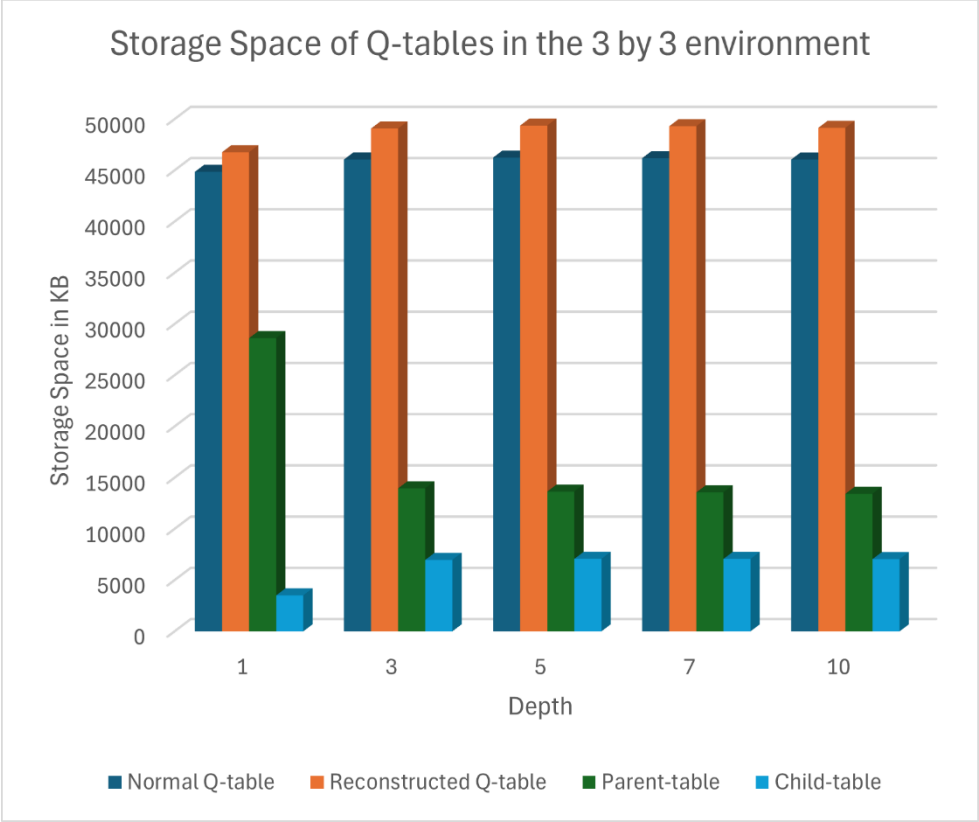

Figure 3 Storage space of Q-tables in 3 by 3 environment

Overall the storage reduction results confirm that the parent child abstraction can reduce the required storage space when enough states can be represented as child states. However the results also show that the abstraction is not automatically beneficial for every grid size and depth and must be planned accordingly.

3.8.2 Child abstraction ratio and Parent to Child ratio

The child abstraction ratio $S'_{abstraction}$ measures how many states of the original Q-table could be represented as child states. It is calculated by dividing the number of children by the original states:

$$S'_{abstration} = \frac{|C|}{|S|} \tag{12}$$

A Value close to 0 means that only a small number of original states were abstracted and a value closer to 1 means that a larger part was abstracted. This work hand in hand with the child to parent ratio P_C which describes the average amount of children represented by one parent:

$$P_C = \frac{|C|}{|P|} \tag{13}$$

During testing the following table was produced for the child abstraction ratio $S'_{abstraction}$:

Table 2 Child abstraction ratio in relation to grid size and depth

Child Abstraction Ratio					
Grid Size	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	0	0,15	0,30	0,35	0,37
3x3	0,36	0,69	0,70	0,70	0,70
4x4	0,03	0,07	0,07	0,07	0,07

Table 2 shows that the child abstraction ratio depends strongly on both the grid size and the depth of the abstraction. The 3 by 3 environment showed once again the strongest abstraction effect. At depth 1, 291400 out of 807151 states were classified as child states, at depth 3 this value increased to 565726 out of 821613 resulting in an increase of 33 percent. Compared to the two by two and the four by four environment this shows that just like the storage reduction it is once again influenced by grid size and depth. The child abstraction ratio reflects the same similarities as the storage reduction percentage. Combining both it shows that an abstraction of 70 percent in the 3 by 3 environment starting with a depth of 3 reflects in a storage reduction of approximately 55 percent.
The child to parent ratio shows the average number of children each parent has:

Table 3 Child to Parent ratio in relation to grid size and depth

Child to Parent Ratio					
Grid Size	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	0	0,17	0,43	0,55	0,59
3x3	0,57	2,21	2,29	2,31	2,33
4x4	0,04	0,07	0,07	0,07	0,06

The child to parent ratio supports the observations made from the child abstraction ratio. In the 2 by 2 environment the ratio increased from 0 at depth 1 to 0,59 at depth 10. This means that each parent represents approximately 0,59 children or every two parents has 1 child. In the next grid size this value is increased drastically as every parent has on average a little more than 2 children. The four by four grid remained at a very low child ratio which goes hand in hand with the previous results. The results show that increasing the PreCalculator depth can improve the abstraction ratio but only up to a certain point. After that point an increase in operation set operations seems more useful. The three provided operations show their limits here.

3.8.3 Policy Agreement and Q-value error

The next testing metric compares the reconstructed Q-table with the original Q-table. This comparison is important because the parent child abstraction is only useful if the reconstructed table preserves the decision behavior of the original Q-table. The metric used to decide if the abstraction works is policy agreement $\pi_{agreement}$:

$$\pi_{agreement} = \frac{\left|\{s \in S : argmax_a\, Q_{original}(s,a) = argmax_a\, Q_{reconstructed}(s,a)\}\right|}{\left|S_{compared}\right|} \quad (14)$$

Policy agreement measures whether the original Q-table ad the reconstructed Q-table from the parent and child table select the same best action for a given state. For each compared state the best action is determined by applying the argmax function to the four Q-values. Table 3 shows the policy agreement for all tested grid sizes and transformation depths.

Table 4 Policy agreement table

Policy Agreement					
Grid Size	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	100%	99,803%	99,784%	100%	100%
3x3	100%	99,655%	99,628%	99,631%	99,644%
4x4	100%	99,835%	99,835%	99,835%	99,834%

The policy agreement is very high for all tested configurations which show that the abstraction works. At depth 1 all three grid sizes reached a policy agreement of 100 percent. This means that for small depths the reconstruction Q-table selected the same best actions as the original Q-table. For higher depth the policy agreement decreases slightly but remained above 99.6 percent which is a good result. These results show that the reconstruction process preserves the learned policies and shows that the abstraction can be applied without loosing information in regards to the best policy.

While policy agreement measures the best action the Q-value error measures the numerical difference between the two tables. The table 5 represents the average Q-value error for each state-action pair.

Table 5 Average Q-error for each state-action pair

Average Q-Error					
Grid Size	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	0	0,1839	0,2218	0,4645	0,4738
3x3	0,1265	0,3044	0,3227	0,3229	0,3230

4x4	0,0024	0,0062	0,0063	0,0065	0,0066

The average Q-value error shows that the reconstructed Q-values are not numerically the same as the original Q-values. This is expected because the child states do not store their original Q-values and only use the parent values in reconstruction. For the two by two environment the average Q-value error increased with depth. At depth 1 the error was 0 which is to be expected as there are no child abstractions. At depth 10 the average error increased to 0.4738. This indicates that more abstraction inevitable leads to more numerical deviation. Compared to the other grid sizes the 4 by 4 size remained very low compared to the 2 by 2 and 3 by 3 environment. It increased from 0,0024 to 0,006 and then stayed at that value. This can mainly be explained by the low child abstraction ratio described above. Since most remained parent states most action-values were stored directly instead of being reconstructed from the parent states. The 3 by 3 environment also stayed stable and reached a max of 0,3230 at depth 10.

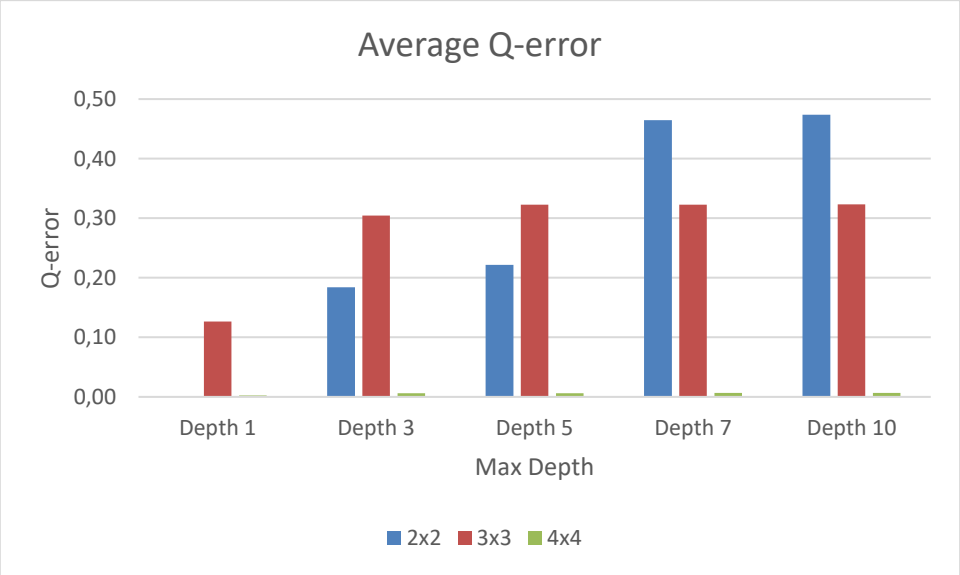

Figure 4 Average Q-value error

The total Q-error is discussed in the appendix I.

3.8.4 Runtime Overhead and Time Relativity

The runtime overhead evaluates how much additional time is required by the splitting algorithm and the reconstruction algorithm in comparison to the normal Q-table. In this evaluation the abstraction overhead consists of the time required for the PreCalculator to determine the best operation sequence for the state and the time the reconstruction algorithm needs to reconstruct the original table. The relative runtime value compares this overhead to the normal training time.

$$T_{overhead} = T_{split} + T_{reconstructed} \tag{16}$$

$$T_{relative} = T_{overhead}/T_{train} \tag{17}$$

A value close to 0 means that the abstraction adds almost no additional runtime. A value of 1 means that splitting and reconstruction require the same time as the training. A value higher than 1 represents that it takes that much more time. A value of 2 for example means that it takes double as long.

Table 6 Time relativity table

Time Relativity					
Grid_Size	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	0,00123	0,00285	0,00335	0,00414	0,00539
3x3	0,59742	0,98558	1,36360	2,31417	3,18643
4x4	2,17796	8,44021	17,75082	35,74774	72,16574

In smaller grid sizes the relative time remained very low for all tested depths. At depth 1 the time only takes up around 0,00123 of the training time and at depth 10 0,00539. In the 3 by 3 environment the time relativity jumps up drastically concluding in approximately 50% of the training time at a depth of 1 and at 3 times the training time at a depth of 10. This shows that anything more than depth 3 would require drastically more time and be therefore useless especially as the quality of abstraction doesn't increase as depicted in the earlier tables. This shows that the abstraction idea suits the medium sized environments as already previously mentioned. For the four by four environment the runtime overhead started with double the time it needed to train at depth 1 and at depth 10 it needed 72 times the training time. This is just too much of a time investment to deem the 4 by 4 environment as anything else but useless for only 6 percent of reduced storage. The experiments show that the runtime overhead increases strongly with grid size. The main reason is the splitting step. A better splitting algorithm would be highly recommended.

The following graph shows the total time of each grid size and the 2 by 2 environment as a time relativity example:

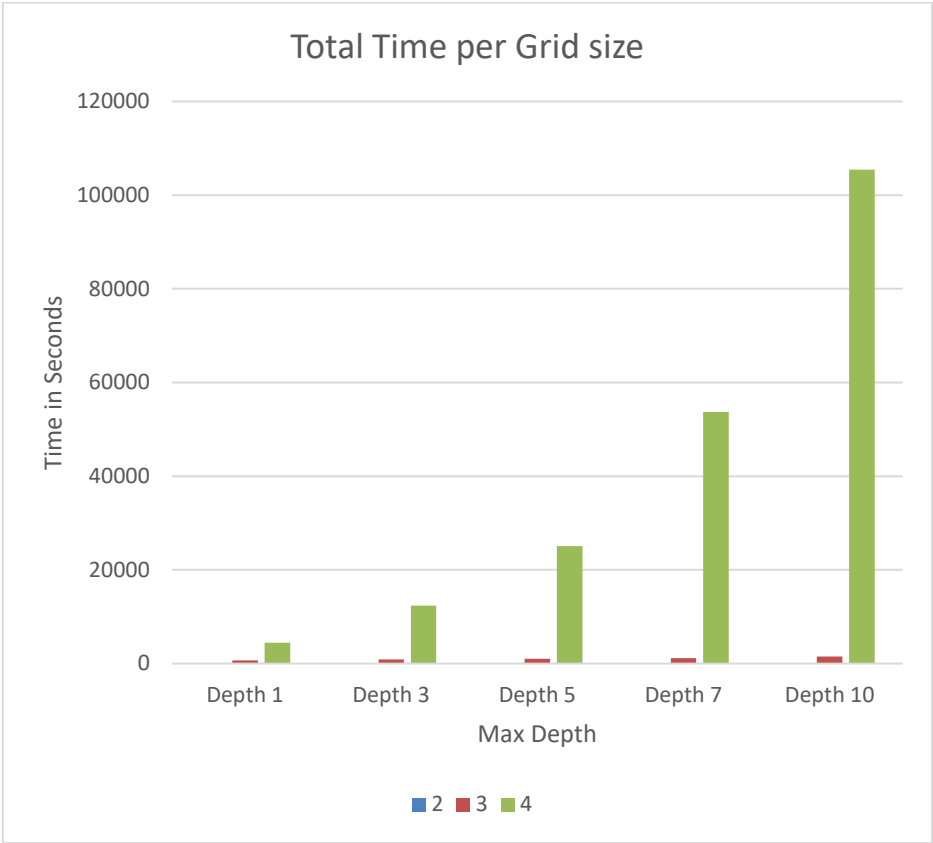

Figure 5 Total time per Grid Size graph

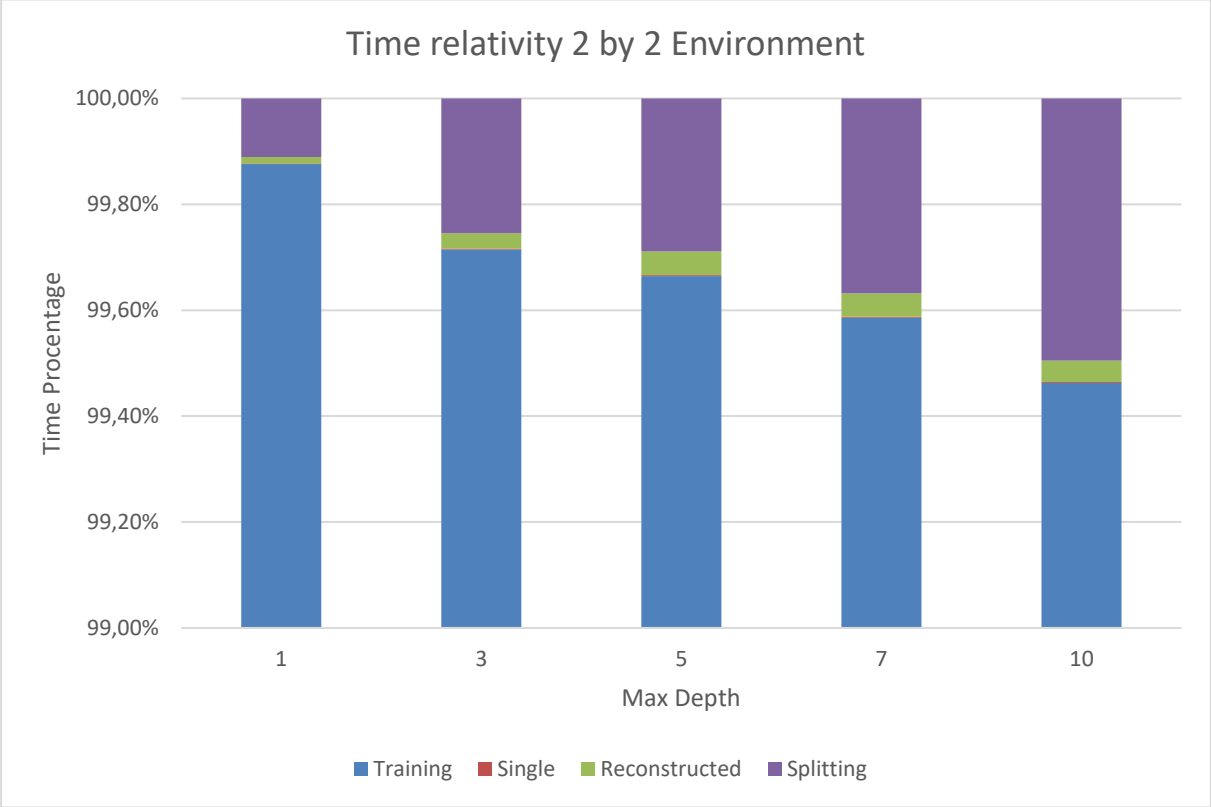

Figure 6 Time relativity 2 by 2 example

3.8.5 Memory overhead

The memory overhead $M_{overhead}$ measures the additional RAM usage caused by the process of splitting parent and child. The average RAM usage between reconstruction, splitting and training was the same and is therefore represented as an average of each step. For each grid size the start memory the end memory and the peak was captured. The peak is especially relevant as it describes the highest observed memory usage during the abstraction.

Table 7 RAM usage table

Ram Usage (MB) Average			
Grid_Size	**RAM_start**	**RAM_end**	**RAM_peak**
2x2	74,11	74,24	74,32
3x3	737,71	740,23	820,39
4x4	8770,27	8777,21	12060,80

The results show that memory usage increased but due to the batch processing was handled by the program without crashing. The increase in RAM usage between grid sizes seems to be because of the larger state spaces as it was the same during training splitting and reconstruction. The largest memory usage appeared in the 4 by 4 environment during reconstruction.

3.8.6 Q-table performance

The last evaluation metric compares the practical play performance of the normal Q-table and the reconstructed table. While policy agreement and Q-value error compare the tables directly on a state-action level this metric compares if both tables behave similarly. For this comparison the agent was run 1000 times for each grid size and for each depth for each table, the reconstructed table and the original Q-table. The scores for each run where then recorded and compared. During this process the results showed that the performance of the two tables were very similar. As already described above the policies where 99,6 % the same which reflected in this test. The better result alternated between the normal Q-table and the reconstructed Q-table. Neither table consistently outperformed the other.

Table 8 Q-table performance table for 1000 tries

Q-table performance						
Grid Size	**Q-tables**	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	**original**	498	503	490	505	496
	reconstructed	502	497	510	495	504
3x3	**original**	512	490	500	511	493
	reconstructed	488	510	500	489	507
4x4	**original**	505	501	498	492	501
	reconstructed	495	499	502	508	499

The results are close to a 50 to 50 distribution for all tested grid sizes. This means that the reconstructed table did not show a systematic disadvantage compared to the original Q-table.

4 Discussion

During this chapter the results of the implemented root-state or parent-state to child-state Q-table abstraction algorithm will be discussed. The main goal of this thesis was to investigate whether a Q-table abstraction can be used to reduce the storage volume of the Q-table needed for reinforcement learning. The results in the previous chapter shows that the approach is technically possible and can reduce the storage usage in some environment but the effectiveness depends strongly on the grid size, the operation depth and the used operation set.

4.1 Evaluation of results

A precise evaluation of the results can be found in appendix J.

The environments showed that a Q-table can be compressed by representing states as children of other states through matrix operations. This approach preserved the learned policy very well as the policy agreement stayed above 99,6 percent in all tested environments. This shows that the abstraction doesn't effect the performance of the agent. The main trade-off is between storage reduction and computational overhead. Higher depths can increase the number of detected children but will result in more runtime. Therefore a higher depth can be said to increase compression ratio or child abstraction ratio. In the 3 by 3 environment it was found out that after a depth of 3 the child abstraction ratio only improved slightly. Policy remained high across all depths and grids. The depth mainly affected compression and runtime. Memory usage increased but not substantially.

4.2 Limitations

The main limitation of this idea is scalability. The results show that the parent child splitting becomes very expensive for larger Q-tables and states. The 4 by 4 environment contains much larger state spaces and this is reflected in the much higher runtime. The next limitation is the implemented operation set. A larger better designed set of operations will likely result in better child abstraction. The final prototype mainly relies on structural similarities but not on contextual similarities. Operations that would detect connections other than structure would reduce the required resources drastically. Another limitation lies in the reconstruction process. The idea behind the parent child abstraction relies on a certain loss of information as the original Q-values of the child states are not preserved. The testing phase was limited by the available computational resources and the implementation of the used agent. A better designed agent and better computer part will result in better performance. The biggest limitation is that in the current prototype with the current operation set the parent child abstraction is limited to structural similarities.

4.3 Generalizability

The proposed parent child abstraction concept itself is not limited to the game 2048 as in principle it can be applied to other reinforcement learning environment where state contain similarities between each other. Grid based games such as 2048 are a natural application are but puzzle games or simplified board games could also be used. The method is more suitable for tabular reinforcement learning rather than deep RL. The results suggest that the abstraction works best when the state space is large enough to contain many repeated structures but not to large that the search becomes impractical. This was the case for the 3 by 3 environment. Therefore the generalizability relies on following conditions:

1. The environment must contain states with structural similarities
2. These similarities must be representable through well defined operations
3. The cost of finding and storing of relationships must be lower than the storage saved by the abstraction

When these rules are applicable the parent child abstraction approach can be useful for other environments.

4.4 Possible improvements

There are several improvements that could increase the scalability of the abstraction idea. The first would be a optimization of the splitting algorithm. Currently the splitting takes up to much time for it to be deemed useable. The next improvement could be a better parent selection strategy. Better operations or a better way to sequence these operations could reduce the space saved drastically. Another improvement could be adaptive transformation depth. Instead of using the same depth for all states one could determine the best depth for each state.

Bibliography

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, Second edition in progress, 2. ed., Cambridge, MA, USA: MIT Press, 2018, [Online: https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf]: A Bradford Book, 2014, 2015.

[2] V. Mnih, K. Kavukcuoglu, D. Silver und et al., „Nature," 2015. [Online]. Available: https://doi.org/10.1038/nature14236.

[3] C. J. Watkins und P. Dayan, „Q-Learning," Kluwer Academic Publisher, Netherlands, 1992.

[4] T. Yang, C. Jin, Z. Wang, M. Wang und M. I. Jordan, „On Function Approximation in Reinforcement Learning: Optimism in the Face of Large State Spaces," Princeton, California, Northwestern, 2021.

[5] M. Szubert und . J. Wojciech, „Temporal Difference Learning of N-Tuple Networks for the Game 2048," in *IEEE Conference on Computational Intelligence and Games, CIG*, Institute of Computing Science, Poznan University of Technology, Poznan, Poland, 2014.

[6] L. P. Kaelbling, M. L. Littman und A. W. Moore, „Reinforcement Learning: A Survey," Journal of Artificial Intelligence research 4, 1996.

[7] S. J. Kazemitabar und H. Beigy, „Automatic Discovery of Subgoals in Reinforcement Learning Using Strongly Connected Components," in *Advances in Neuro-Information Processing*, Berlin, Heidelberg, 2009.

[8] K. Gupta und H. Najaran, „Exploiting Abstract Symmetries in Reinforcement Learning for Complex Environments," in *2022 International Conference on Robotics and Automation (ICRA)*, Philadelphia, PA, USA, 2022.

[9] D. Abel, D. Arumugam, L. Lehnert und M. L. Littman, „State Abstraction for Lifelong Reinforcement Learning".

[10] L. Li, T. J. Walsh und M. L. Littman, „Towards a Unified Theory of State Abstraction for MDPs," Department of Computer Science Rutgers university, Piscataway.

[11] C. Lu, Z. Chen, C. Wu und A. Wierman, „Overcoming the Curse of Dimensionality in Reinforcement Learning Through Approximate Factorization,“ 01 May 2025. [Online]. Available: https://openreview.net/forum?id=aEsIW59zDm. [Zugriff am 07 04 2026].

List of Figures

List of Tables

List of Abbreviations

ML	Machine Learning
RL	Reinforcement Learning

Documentation table of AI-based tools

AI-based tools	Intended use	Prompt, source, page, paragraph...
ChatGPT (5.0)	Grammar and spelling	“Can you show me grammatical errors in the following Paper [whole document]”.
Google Translate	Translation	Translation of specific phrases and words.

A: Reinforcement Notation Continuation

s_t	$state\ at\ time\ t$
s'	$next\ state$
a_t	$action\ at\ time\ t$
a'	$next\ action$
r_t	$reward\ at\ time\ t$
R	$total\ Reward$
$A(s)$	$set\ of\ all\ possible\ actions\ given\ state\ s$
$R(s,a,s')$	$reward\ funtion$
G_t	$return\ (cumulative\ discounted\ reward)$
$P(s'\|s,a)$	$transition\ probability$
π	$policy, decision\ making\ rule$
$\pi(s)$	$action\ taken\ given\ state\ s\ under\ deterministic\ policy\ \pi$
$\pi(a\|s)$	$probability\ of\ taking\ action\ a\ given\ state\ s\ under\ stochastic\ policy\ \pi$
$\pi^*(s)$	$optimal\ policy, best\ action\ with\ highest\ expected\ return$
$V^\pi(s)$	$state\ value\ function, expected\ highest\ return\ given\ state\ s\ under\ policy\ \pi$
$V^*(s)$	$state\ value\ function\ (optimal\ policy), expected\ highest\ return\ given\ state\ s\ under\ optimal\ policy\ \pi^*$
$Q^\pi(s,a)$	$action\ value\ function\ (\ Q-Function\ ), expected\ highest\ return\ given\ action\ a\ in\ state\ s\ under\ policy\ \pi$
$Q^*(s,a)$	$action\ value\ function\ (\ Q-Function\ ), expected\ highest\ return\ given\ action\ a\ in\ state\ s\ under\ optimal\ policy\ \pi^*$

The following Notation is specifically for the abstraction algorithm discussed during this thesis:

$C(p)$	$set\ of\ all\ child\ states\ for\ root/parent\ state\ p$
$O(c)$	$set\ of\ all\ possible\ operations\ for\ child\ state\ c$
$O(p)$	$set\ of\ all\ possible\ operations\ for\ parent\ state\ p$
$M(n)$	$memory\ usage\ of\ non\ abstracted\ Q-table\ for\ grid\ of\ size\ n$
$M'(n)$	$memory\ usage\ of\ abstracted\ Q-table\ for\ grid\ of\ size\ n$
$T(n)$	$time\ usage\ of\ non\ abstracted\ Q-table\ for\ grid\ of\ size\ n$
$T'(n)$	$time\ usage\ of\ abstracted\ Q-table\ for\ grid\ of\ size\ n$

B: Ideal Solution Continuation

The splitting of parent and child states should ideally occur during training. The agent chooses an action the environment reacts with a new state and the reward. The agent should ideally recognize the new state as a pre-existing state in its knowledge base and map it to an existing parent. If not, the state should be marked as a parent and be saved as such. The recognition process can be done in multiple ways. A simple breadth-first tree search will yield the best results. In the ideal solution this would be the way. Another way is a depth-first search or a list of existing states that are already known to be able to be derived from the parent. This pre-existing list will later be used in the implemented solution as it combines fast reliable lookup without too much compute Overhead. Sacrificing results for performance. The list can be created during runtime or before the program is run. In order for the list to be created during runtime the agent chooses a known parent state and calculates all possible states reachable from that parent in a specific time or depth. These possible reachable states are then saved in cache with the index of the parent and the operations to reach that state. Later when the agent comes across a state it first checks with the already calculated states and if it finds a match it copies the operations and the index of the parent state to the now child state.

With the ideal search algorithm of the breadth-first search the Q-table could ideally be divided into a small parent Q-table and a larger child-operations table. The memory usage of both tables together should be smaller than the original Q-table as the child only saves keys to states and instructions. The reduced space is determined by comparing the storage size of the original Q-table with the storage size of the parent and the child table:

$$M' = M_P + M_C \tag{4}$$

The storage reduction can be expressed as:

$$M_{reduction} = 1 - M'/M \tag{5}$$

Where $M_{reduction}$ describes the relative reduction in storage space. A value of 0 would mean that no storage was saved. A value close to 1 would indicate a strong reduction in storage. The value of $M_{reduction}$ can never be exactly 1 as that would mean the abstraction takes up no space. In the ideal solution $M_{reduction}$ should have a value of 0,5. This would indicate the reduction of storage space by 50 percent. The abstraction is only beneficial if the combined size of the parent and child tables is smaller than the original Q-table:

$$M' < M \tag{6}$$

This would mean that the storage saved by removing complete child state-action pairs must be larger than the additional overhead introduced by storing parent indices and operation sequences. Larger operation instruction sets with more possible operations would also increase

the storage reduction. In the ideal solution operations such as edge expansion, chunk detection and triangle detection would reduce the required storage space exponentially.

Implemented Operations

A state can be rotated by 90° increments. Two states are equivalent if one can be derived from the other if a rotation is applied. Matrix A and matrix B are the same if you can rotate A to get to B. This operation exploits the symmetry of grid-based worlds as the action up can also be rotated to the left, down or right. In the child Q-table a rotation is marked with the character “r”. During the reconstruction process the operation sequence is parsed and the operations are applied in sequence.

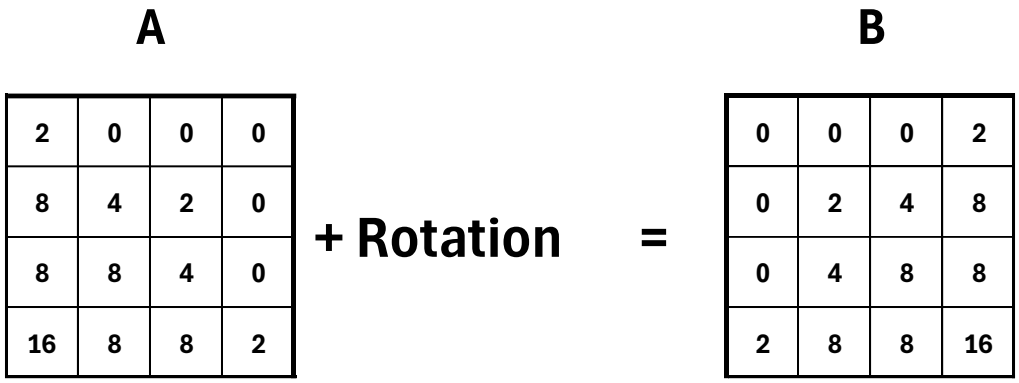

Figure 7 Transformation-based rotation example

The action values are remapped according to the rotation during the reconstruction process. For a 90 degree rotation the action-values change as the following:

$$new\ up = old\ right$$
$$new\ left = old\ up$$
$$new\ down = old\ left$$
$$new\ right = old\ down$$

A state can be scaled by multiplying all tile values by a constant factor. Two states are equivalent if one is a scaled value of the other. Matrix A and matrix B are the same if you can double A to get to B. This is particularly effective in games like 2048 due to the exponential structure of the tile values. In the child table a scaling or doubling of the values is characterized by the character “d”.

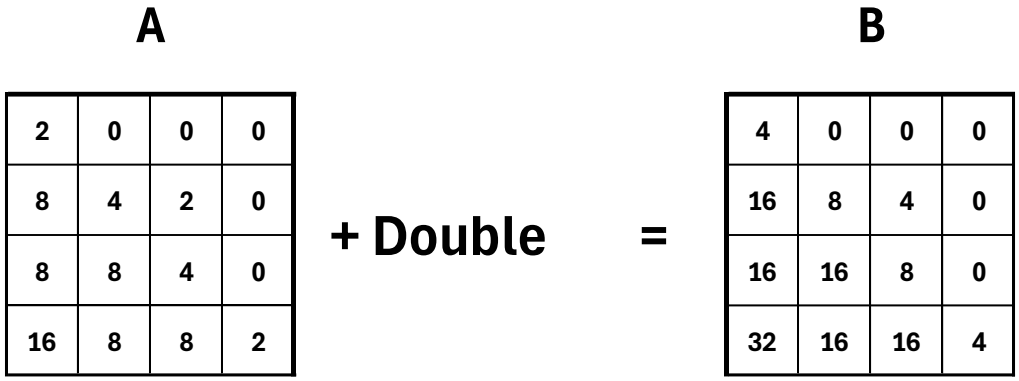

Figure 8 Transformation-Based scaling example

Unlike rotation and mirroring doubling does not affect the structural orientation of the board and is therefore not applied to the actions.

A state can be mirrored along horizontal or vertical axes. Two states are equivalent if one is a mirrored version of the other. Matrix A and matrix B are the same if you can mirror A to get to B. In the code mirroring is split into two. Horizontal mirroring is marked with the combined character “mh” and vertical mirroring with “mv”.

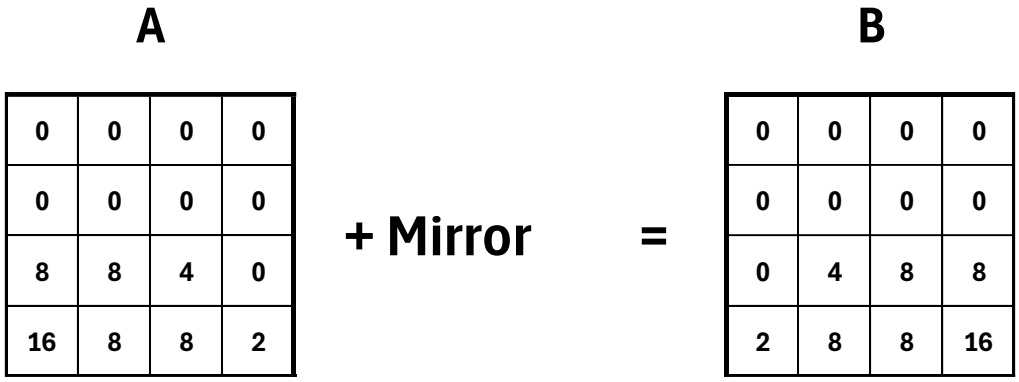

Figure 9 Transformation-Based mirroring example

Horizontal mirroring swaps the left and right sides of the board. Therefore the Q-values of the actions left and right are swapped as well in order to assure structural correctness. While in Vertical mirroring the upper and lower actions are swapped. Mirroring is useful as many 2048 board patterns can appear in reflected form. During testing it has been shown that rotation was used the most followed by mirroring equally often on both axes. Scaling the matrix has proved to be less useful as it only occurred a fraction of the time.
Additional operations and concepts such as edge expansion, chunk detection and shape-based pattern recognition were considered during the ideal solution but proved to hard to implement in the given time. While rotation and mirroring have clear action mappings, operations such as edge expansion or chunk detection do not provide an equally direct transformation of

the Q-values. Therefore, they would require additional rules or learning-based correction mechanisms to avoid excessive information loss. The non implemented operations are discussed later as possible improvements and as ideas for a master's thesis.

Non-implemented ideal operations

Edge expansion is the concept of recognizing two state as the same state if one would add a line of twos to every edge. This would ideally be done without compromising the original action-

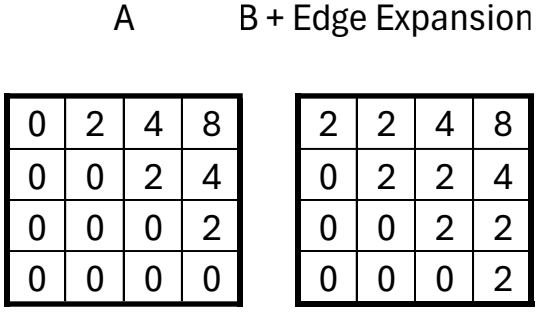

Figure 10 Edge Expansion example in the ideal Solution

values such that the state with an edge and the state without the given edge have the same best action. In the implementation this cannot be done as a new set of values changes the policy to drastically.

Chunk detection would need a form of correlation detection where the agent recognizes two state to be the same if two or more chunks are equal. In the ideal solution chunk detection is instantaneous and can be done with different sizes of chunk where a chunk can overlay with many other chunks. One can than save a state not as a grid of four by four tiles but as a connection of chunks that are present in a singular or multiple parents. This approach could not only be used for child abstraction but also deem useful in abstracting the parent Q-table as well.

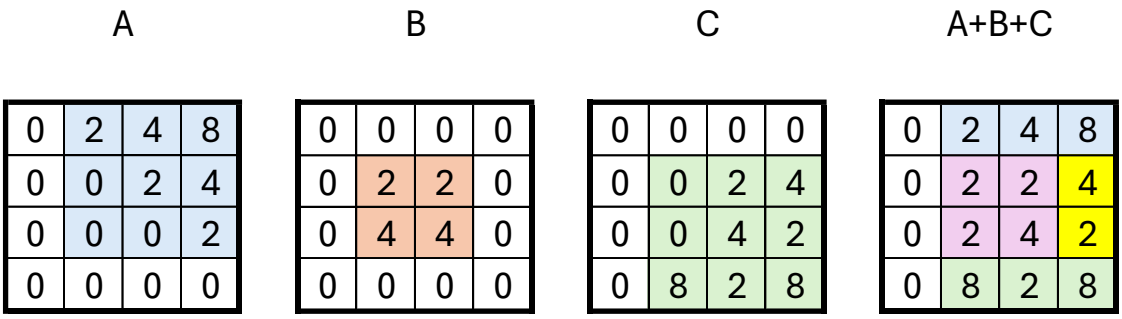

Figure 11 chunk detection and reconstruction visualized in the ideal solution

The next theoretical approach described is triangle detection. It combines the idea of chunk detection with geometrical forms such as triangles, pluses and minus. The chunking would no longer be locked to squares but can also be applied to chunks of different forms. This would reduce the number of states but is still only available in the idealized solution as just like chunk detection the abstraction needed to be applied to the action-values is to much for this thesis.

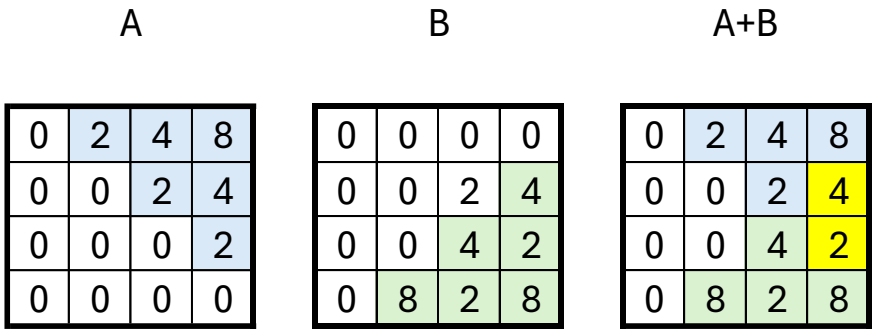

Figure 12 triangle detection example in the ideal solution

C: Development iterations and algorithmic complexity

The development process followed an iterative prototyping approach. First the environment was implemented. In order to ease testing and for convenience the game 2048 was created in such a way that the world can be created with different sizes. The original game is played on a four by four grid but in this implementation three by three and two by two are also possible. The environment was written in python as a class that can be called by the agent to facilitate the training. It provides the game logic required for the training of the agent. On first call the environment creates the grid. Before every action it provides valid actions and checks the grid

for changes. It handles tile movement, merging, reward calculation, random tile spawning and terminal states. The game environment was essential for further development therefore much care was taken in its creation. The environment was the only part that did not need constant revisions.

The rest of the implementation was created in several phases. Each phase produces an intermediate result that was tested and used as the input for the next phase. The first phase was the creation of the Q-learning agent. The agent observes the environment. It looks at the current state and selects an action using an epsilon-greedy strategy. This strategy balances exploration and exploitation. It checks epsilon against a random number between zero and one. If the number is smaller than epsilon a random action will be chosen. If not the agent searches for the state in its Q-table. If a match is found the best Q-value is chosen via the argmax function. It returns the action in its Q-values with the highest value. If the state does not exist in the Q-table a random action is chosen. The epsilon will then be reduced by a predefined value. A high epsilon value forces the agent to chose random moves and a small epsilon value forces the agent to exploit its existing knowledge base. After the agent choose an action it sends this action to the environment via the step function interface. This function is the main function in the environment it takes an action as argument and returns the next state and the reward gained from that move. The agent then updates its Q-table with the results it got from that action. This is where the next phase is implemented.

The second phase focused on the design and implementation of transformation operations. These operations where required for the agent to determine if two states are child and parent. It is easy for a human to determine if two states are nearly identical but hard for a computer. The functions are part of a class currently called “PreCalculator” which consists of operational transformation function mapped to action transformation functions. The class is called by the agent by providing the parent state as argument and receiving a list of reachable state-operation pairs. This interface provides the ability for the agent to determine if one state is a child of a parent state.

A

0	2	4	8
0	0	2	4
0	0	0	2
0	0	0	0

B

0	4	8	16
0	0	4	8
0	0	0	4
0	0	0	0

Figure 13 comparison between two similar states

The first version of the abstraction algorithm followed the ideal solution. The first state in the original Q-table was selected as a parent state. Each following state would then be compared directly to each parent state starting with the first. If the parent state and the to be determined state where not equal the to be determined state would be operated on in a breadth-first search algorithm where every possible operation was mapped to a branch. The agent would follow every branch until a specific depth was reached. If the state did not match after every possible operation sequence the to be determined state would be counted as a parent state and the next state was tested. If the state matched to the parent state it received the index of the parent and the list of operations applied to get to the parent and be classified as a child. This version was useful for validating the theoretical concept because it searched for parent child relationships directly and could therefore achieve a high degree of accuracy. The two by two version of the game environment also proved useful as it reduced the time needed drastically. In theory this approach proved close to the previously discussed ideal solution. However the first version did not scale well. For small Q-tables and small states the runtime was acceptable but steadily increased for each new state. For larger Q-tables the number of comparisons increased rapidly as each new state had to be compared to a growing list of parent states. In addition each comparison required a breadth-first search over possible operation sequences.
In Big-O-Complexity this version can be described as the following:

$n = total\ number\ of\ states\ in\ the\ Q - table$
$p = number\ of\ parent\ states$
$d = \max operational\ depth$
$k = number\ of\ allowed\ operations$

Average case:

$$O(n * p * k^d) \tag{7}$$

Worst case $p = n$:

$$O(n^2 * k^d) \tag{8}$$

The next version introduced batch processing. Instead of processing the entire Q-table the Q-table was chunked into smaller batches. This reduced the memory usage drastically and allowed it to process larger files and state spaces. The three by three environment version helped here as it still created large state spaces of hundreds of thousands of states but still in small enough state sizes of only 9 values to be compared. The batch process loaded only small state spaces into memory which improved practical feasibility but it also showed the limitation of the approach. Only a subset of state is processed at the same time so some parent child relationships may never be found. This reduced the achieved score as the corresponding parent state might be located out of the current batch. Therefore batch processing represents a trade off between abstraction and computation. In the ideal solution, batching is not necessary. With different machine learning models an ideal solution might be feasible if a classification model can be trained to distinguish between parent and child. This could be a topic for a masters thesis.
The Big-O-Complexity of this approach is as followed:

$b = batch\ size$

$$O(n * b * k^d) \tag{9}$$

If b is a constant size then it grows linearly:

$$O(n * k^d) \tag{10}$$

To further improve the runtime a pre-calculator was introduced. This is the current version of the implementation. Instead of comparing every new state with existing parent states and running multiple breadth-first searches the pre-calculator generates reachable child states for a given parent state and saves these in a cache. It runs the breadth-first algorithm but saves every state. These states are saved as the index of the parent and the operational sequence to get to that parent. The reachable states are saved in a list resulting in an $O(1)$ lookup time. During the splitting process a to be determined state can then be checked against the precomputed lookup table. The state is a parent if it does not exist in that table and a child if it does. This approach significantly reduced the number of repeated computation.
In Big-O-Notation:

$$O(p * k^d + n) \tag{11}$$

In the ideal solution the agent would split the state after receiving the reward from the environment. But during this development process it was found out that this increases the processing overhead substantially. Therefore, it was decided that the splitting will be done at the end of all episodes and in batches. The idea behind batches was to reduce RAM usage as running

through an entire Q-table and comparing all states took up to much memory space. In testing a large Q-table with millions of states easily exceeded 64GB RAM usage causing the testing environment to crash. The output of the splitting phase is a parent table containing complete state-action pairs and a child table containing parent references and operation references as well as the best action determined by the agent via a argmax function. It saves the highest Q-value of each state of the child states as the best direction for later policy accuracy.

The reconstruction step is done at the beginning of the training or play function of the agent. It combines the parent and child tables into the original Q-table. During reconstruction each child entry is combined with its referenced parent state. The stored operation sequence is applied to reconstruct the original state from the parent state. This is done via an interface from the "PreCalculator" class. In order to reduce loss of information the actions need to be transformed together with the state. This is especially important for orientation of the board such that rotation and mirroring can be done. A rotated state must also have the same rotated actions. If a state is rotated the meaning of the actions up, down, left and right changes as well. Therefore the same operational sequence used for state reconstruction is also applied to the action values. Without this step the reconstructed state could receive action values that refer to the wrong directions. The reconstruction step therefore has two return values. The reconstructed state generated by stored operations to the parent state and the reconstructed action state list generated by transforming the parent action values. These outputs are then combined into state-action pair and inserted into the reconstructed Q-table. The reconstructed table is then used for further training. This phase was necessary because the parent-child representation is only useful if it can be converted back into a format that the Q-learning agent can use later. The Q-table can then be used for continued training or for playing the game. The reconstructed table is later evaluated against the original Q-table int the evaluation phase.

D: Testing and Evaluation

$M_{reduction}$	$Storage\ reduction$
$S'_{abstraction}$	$child\ abstraction\ ratio$
P_C	$child\ per\ parent\ ratio$
$\pi_{agreement}$	$Policy\ agreement$
Q_{error}	$Q-value\ error$
$T_{overhead}$	$Runtime\ overhead$
$M_{overhead}$	$Memory\ oberhead$
$Q_{average}$	$best\ average\ Q-table$

The storage reduction $M_{reduction}$ compares the size of the original Q-table with the combined size of the reconstructed Q-table. As previously mentioned in chapter 2.1 the storage reduction can have a value between zero and one where 0 would indicate no storage reduction and 1

extreme reduction. This metric is used to identify the usefulness of the abstraction approach it determines if the idea can be used to save space. The storage reduction is formed via the given equation:

$$M_{reduction} = 1 - M'/M \tag{5}$$

Where M' is the combined storage space of the parent and child tables:

M_P $\quad storage\ space\ of\ the\ parent\ Q-table$
M_C $\quad storage\ space\ of\ the\ child\ Q-table$
M' $\quad storage\ space\ of\ the\ combined\ parent\ and\ child\ table$

$$M' = M_P + M_C \tag{4}$$

The second evaluation metric is the child abstraction ratio $S'_{abstraction}$. It describes the possible number of states the original Q-table could be represented as if the abstraction approach is used instead of storing it as complete state-action pair. It is used to determine the effectiveness of the abstraction algorithm at identifying structural similarities between states.
Let $|S|$ be the total number of states in the original Q-table and $|C|$ be the total number of states classified as children via the previously described splitting algorithm. The children abstraction ratio is defined as the following:

$$S'_{abstration} = \frac{|C|}{|S|} \tag{12}$$

The child abstraction ratio $S'_{abstraction}$can have a value close to zero meaning that only a small number of states could be abstracted as children or a value close to one which would indicate that most states could be represented as child states. The value can be zero if no children could be found but can never be one as it would suggest no parent states present. Therefore, a higher value indicate that the operations were able to identify more structural similarities in the Q-table. It can also be said that more different operations can also lead to a higher child abstraction ratio. Therefore splitting the original Q-table with only rotation and doubling as operations will always have a lower child abstraction ratio than if more advanced instruction are included. Operations such as edge expansion and chunk detection previously described in this thesis would ideally increase the value closer to 1.

The child-to-parent ratio P_C compares the number of parent states with the number of child states. It describes the structure of the abstracted Q-table. A small number indicates that the number of state-action pairs for the given Q-table can be represented through a smaller number of parents via the abstraction method. Here $|P|$ is the number of parent states and $|C|$ the number of child states. The ratio is defined as:

$$P_C = \frac{|C|}{|P|} \tag{13}$$

A high value indicates that on average each parent has a higher number of children.

The fourth evaluation metric is policy agreement $\pi_{agreement}$. It measures whether the original Q-table and the reconstructed Q-table choose the best same action for a given state. Since the goal of the abstraction is not necessarily to reconstruct all Q-values numerically identically preserving the best action via the best direction marker in the child table is the most important criterion for maintaining the behavior of the agent. For each state s the best action is determined by the argmax function previously described. The policy agreement is then calculated by comparing the best action in the original Q-table with the best action in the reconstructed Q-table:

$$\pi_{agreement} = \frac{\left|\{s \in S : argmax_a\, Q_{original}(s,a) = argmax_a\, Q_{reconstructed}(s,a)\}\right|}{\left|S_{compared}\right|} \tag{14}$$

Here $S_{compared}$ is the set of states that exits in both the original and the reconstructed Q-table. Ideally $S_{compared} = S$. A value of 1 means that both Q-tables select the same best action for every state given. A value of zero means that the two Q-tables have drastically different policies and that not state hast the same best action.

The next metric is the Q-value error Q_{error}. As the name implies this metric measures the numerical difference between the Q-values of the original Q-table and the reconstructed Q-table. While policy agreement only checks for the best action this metrics shows how much numerical information was changed during the abstraction process. For each compared state the absolute difference between both the original and the reconstructed Q-values is calculated for all actions. The average Q-value error is defined as:

$$Q_{error} = \frac{1}{\left|S_{compared}\right| * |A|} * \sum\nolimits_{s \in S_{compared}} \sum\nolimits_{a \in A} |Q_{original}(s,a) - Q_{reconstructed}(s,a)| \tag{15}$$

A is the set of possible actions provided by the environment. A value of zero means that the reconstructed Q-table Q-values are identical to the original Q-values. This can never occur as the concept of this abstraction sits on the idea that the actions of a state with a similar structure can be used by states of the same structure. Therefore the original actions are discarded and are reconstructed from the actions of the parent. The idea behind this metric is to find the size of the reconstruction error as a higher value indicates more errors in reconstruction.

The sixth and seventh evaluation metrics are runtime overhead $T_{overhead}$ and memory overhead $M_{overhead}$. The runtime ahead measures the additional required time by the abstraction

process and the memory overhead the additional RAM requirements during the abstraction process. The time overhead includes the time required for the splitting of the original Q-table into parent and child state as well as the time required for the reconstruction of the Q-table from these tables. It is defined with the following variables where T_{train} is the required time for the normal Q-learning training without abstraction, T_{split} the time required for the parent-child splitting and $T_{reconstructed}$ for the time needed to rebuild the parent and child tables into the reconstructed Q-table. The total abstraction overhead can therefore be defined as:

$$T_{overhead} = T_{split} + T_{reconstructed} \tag{16}$$

In order to compare the runtime overhead to the normal time the following equation is required:

$$T_{relative} = T_{overhead}/T_{train} \tag{17}$$

The runtime overhead is important as it provides information on the usefulness of the abstractions. In order to reduce the runtime relative to the normal training the depth of the splitting algorithm can be reduced or the number of possible operations. This will inevitable lead to less abstraction. One must make a compromise between speed and storage space. The same goes for memory overhead as more operations require the agent to keep more states in cache which will lead to more RAM usage. Let M_{normal} be the memory usage of the normal Q-learning agent and $M_{proccess}$ be the memory maximum memory usage during the abstraction. The additional memory overhead can then be expressed as:

$$M_{overhead} = M_{process}/M_{normal} \tag{18}$$

Here the memory overhead relative to the normal RAM usage can be expressed as:

$$M_{relative} = M_{process}/M_{normal} \tag{19}$$

A high memory overhead would indicate that the abstraction method may not scale well to larger Q-tables. This happened during testing as the first version of the splitting algorithm resulted in the program to crash. A better way of reducing this problem can be achieved by saving each batch in a temporary file that is then deleted in the end of the program. In the implementation of this thesis this was done using a SQLite implementation where the reachable dictionary was periodically written to and compared against in order to keep the program from crashing.

The last evaluation metric is the $Q_{average}$ metric it averages the best results during the play stages of the agent and outputs the best Q-table given the original and reconstructed table. In an ideal solution the best table should switch between original and the reconstructed Q-table as the reconstructed table should be equal to the original Q-table. During testing this was also

found out as the best direction parameter in the child table nudged the results to force the best policy. Resulting in the two tables to be equally as good in already known state spaces.

E: Tool selection explained

The tools for the implementation were chosen based on the requirements. The goal of this thesis is the development and evaluation of an experimental Q-table abstraction algorithm therefore the language python seemed the best pick as it is fast for prototyping, specialized for machine learning, supports matrix operations and has the ability to process larger Q-tables. An alternative would have been C++ as it offered better runtime performance and more direct memory control. However the implementation effort additionally required for C++ made it unsuitable for prototyping. Pythons easy to understand syntax and readability proved a big positive during this thesis as it made it easy to learn the language and be able to work on it over the course of months. Since the goal of this paper is to evaluate the feasibility of the abstraction concept rather than to build a production ready high performance system python seemed to make more sense.

NumPy also seemed a obvious choice for representing transforming game states. As it is a C based library meaning the implementation ran faster than a normal python code. This helped during the testing phase as it speed up the time needed to finish each test. A board of 2048 can naturally be represented as a matrix and the operations as NumPy arrays. This made NumPy also suitable for testing transformation based state abstraction. Without NumPy these matrix transformation would have to be implemented manually. This would have increased the code complexity and the risk of errors in the implementation.

In order to view the results in a human friendly format the states were saved primarily as CSV files. This was also a reason python was chosen as C++ does not include a CSV library as standard. The CSV files saved the original Q-table, the parent table, the child table and the reconstructed Q-table. During development this was useful as it eased the ability to compare individual states. In production this should be replaced with python pickle files or a different better suited file system as CSV are good for readability but not storage capabilities.
As already mentioned for larger Q-tables a SQLite implementation was introduced to reduce the amount of RAM needed when processing large Q-tables.
The development environment was Visual Studio Code as it was already known and provided enough tools needed for this thesis.

F: Background on the game 2048 and Q-learning

This section provides information for the game 2048 and Q-learning. This is necessary in order to understand the implemented solution. It will begin by briefly explaining the game and its rules and then be followed by an explanation of Q-learning.

Background: 2048

The game 2048 was created by Gabriele Cirulli on march 2014. It is a single player, grid based puzzle game were the aim is to achieve the highest score possible by combining tiles with the same value into tiles of the combined value. It is played on a 4 by 4 matrix where every tile starts with the value 0. The game loop is such that after every player action a tile will be created on an empty tile. A tile is counted as empty if its value is 0. The player can choose a direction up, down, left and right. After choosing one of these direction every tile with a value higher than 0 will move in the chosen direction. A tile will stop, if it hits a tile of the same value and will then merge with that tile, if it hits one of the 4 walls of the world or if it hits a tile with a value higher or lower than its own value. Tile of value 0 do not move and just count as empty tiles therefore can be traversed by other tiles. After all tiles moved a new tile is created at a random empty space in the grid world and the player can choose their next move.

0	0	0	2
0	2	2	0
0	4	0	8
0	0	0	0

Action: Right

0	0	0	2
0	0	0	4
0	0	4	8
2	0	0	0

Figure 14 Example of new game state after player moved right

A new tile can have a value of 2 with a ratio of 9 to 10 or a value of 4 with a ratio of 1 to 10. Every tile can have a value of the power of two. For example a tile can have the value 0 if its an empty tile or multiple of 2 such as 2, 4, 8, 16 and so forth. When a tile collides with a tile of the same value they merge into one tile with twice the value. For example two tiles with the values 4 will merge into a singular tile of value 8. The game ends if no valid move is possible. The goal of the player is to achieve the highest score. The score is calculated by adding together the values of all tiles on the field.

The simplicity of the game leads to it being commonly used for reinforcement learning. Each player input produces a new state s. In context of RL the agent chooses an action a from the set of possible actions given the state [5] [1]. This interaction naturally forms a Markov Decision Process [6]. A challenge in applying reinforcement learning to 2048 is the vast amount of state space [5] [4]. The number of possible grid configurations grows exponentially due to the different tile values and their positions [11]. As a result, storing every state and every action in a Q-table becomes memory expensive very quickly.

In this thesis 2048 is used as the environment for the Q-learning agent. Multiple versions of this environment are implemented. A two by two version a three by three version and the standard four by four version.

In policy-based RL the interaction between environment and agent form a Markov Decision Process (MDP).The MDP defines the environment, while the policy determines which action to take in each state. The chain begins in state s_0. The agent observes the state and selects an action a_0. In the case of the game 2048 an action can be any of the 4 cardinal directions up, down, left and right. This action is send to the environment and the environment changes based on the action. This leads to the state s_1. A reward r_1 is sent back to the agent to help him determent the next move. The agent observes the state again and chooses an action. This leads to the Markov decision chain:

$$s_t \rightarrow a_t \rightarrow r_{t+1} \rightarrow s_{t+1} \rightarrow \cdots \tag{20}$$

A key concept of the MDP is that each state only depends on the immediate state and action before it and not on every state before it. This can be derived from the transition probability:

$$P(s_{t+1}|s_t, a_t) \tag{21}$$

The objective of an policy-based agent is to learn a policy π which maximizes the expected sum of possible returns:

$$G_t = \sum_{k=0}^{e} \gamma^k r_t + k + 1 \tag{22}$$

In the context of 2048 e is the number of episodes the agent is playing which can range from 1 to 100.000. In normal MDP it is assumed that e is ∞. The discount factor γ is a number between 0 to 1 (starting with 1) and ensures that the agent prioritizes exploration first and exploitation later. In this implementation a value-based approach was chosen.

Background: Q-learning

Machine Learning (ML) is a way for computers to understand data and or environments and apply specific actions given a certain input. ML divides itself into 3 sub-categories, Supervised learning where developers use labelled data to train A.I. models for predictive tasks. The data is split into a training set and a test set and the A.I. is trained. In unsupervised learning the data is unlabeled and the A.I. is forced to find patterns or clusters in the data. The result is harder to validate and the way the A.I. found the answer can be hard for developers to understand, such as in neural networks.
Other than in supervised learning and in unsupervised learning in reinforcement learning (RL) the A.I. or in RL, often called agent learns not through data but through an environment, often simplified, through trial and error. Guided only with rewards and penalties, the agent will find the best way through a maze or get the highest score in a game. It is often used in robotics or

autonomous systems such as self-driving cars where agents must adapt over time. In these circumstances the agent chooses the best possible path with help of deep learning to avoid accidents on the road.
Q-learning is such an RL approach. The agent interacts with an environment by receiving a state s, based on that state selecting an action a and receiving a reward r based on the chosen action. This loop is called the Reinforcement loop.

In the context of 2048, a state is a snapshot in time of the grid and the action is one of the movement directions up, down, left and right. Q-learning is a value-based approach, meaning its goal is to learn the optimal action-value function $Q^*(s,a)$ instead of the optimal policy as described in the MDP. This function estimates how useful it is to choose action a in state s. During training the agent continuously updates its Q-table. This table consists of state-action pairs where every state has corresponding action-values. It updates this Q-table based on the rewards it received from the environment and the estimated
Value of the next state.
The Q-learning update rule is as followed:

$$Q(s,a) \leftarrow Q(s,a) + \alpha * (r + y * \max'_a Q(s',a') - Q(s,a)) \tag{23}$$

The learning rate α determines how much new information overrides old information. This is used in order to control the stability and speed of a given agent. A high learning rate of 1 results in an agent that discards all previous estimates and relies solely on new experience and a low value of 0 results in no new experiences. Depending on the requirements one must decide between stability with the learning rate close to zero or rapid adaptation to new environments with a learning rate close to 1. In this implementation and in many others the learning rate is a constant set at 0.1 for simplicity.
The immediate reward r represents the reward received from the environment after every step. It is often regulated with the discount factor gamma γ to prioritize future rewards instead of instant gratification. The discount factor is a number between 0 and 1.
The function $\max'_a Q(s',a')$ is the highest estimated value of the next state. The function selects the highest possible action a or the value with the maxmimum value and chooses that move.
The implemented agent uses a simplified update rule where the discount factor is inexplicitly set as 1. Therefore the estimated value of the next state is not discounted and future rewards are treated with the same weight as immediate rewards. This was chosen as the focus of this thesis is not the optimization of the Q-learning agent but the development and evaluation of the parent and child Q-table abstraction. This simplified rule makes generating Q-tables easier to compare during abstraction and reconstruction:

$$Q(s,a) \leftarrow (1-\alpha) * Q(s,a) + \alpha * (r + \max'_a Q(s',a')) \tag{24}$$

Additionally a simplified update rule was also chosen as it creates small immediate regards which are useful in order to keep the board open for later moves. However the absence of an explicit discount factor can prove a limitation in production environments in which case a high gamma value of 0.9 is recommended.

The agent decides its moves based on an epsilon-greedy strategy. With probability ε the agent selects random valid actions and with a probability of $1-\varepsilon$ actions in its Q-table. It selects these actions in its Q-table by comparing the Q-values and selecting the action among it with the highest Q-value. During the training loop the agent trains on episodes where every episode is one training session. At the end of each episode the epsilon value is reduced by a specific decay value. This shifts the agent from exploration to exploitation. As a large epsilon value results in the agent exploring random actions and a small epsilon value in it using its existing knowledge base.

The Q-table stores an entry for every observed state. In this thesis a state is a flattened NumPy matrix saved as a tuple. The Q-values for each state are a list of the four actions.

$$Q(s) = ((state), up, down, left, right) \tag{25}$$

For example a two by two board:

$$\begin{vmatrix} 0 & 2 \\ 2 & 4 \end{vmatrix}$$

Is represented as the tuple:

$$(0, 2, 2, 4)$$

This representation allows the state to be used as a key in a dictionary.
Parent:

```
index;state;up;down;left;right
0;2,2,0,0;0.0;31.465654068747885;26.9191467740004;26.28482026848514
1;0,2,0,2;29.315083246115393;30.85967688016068;28.237153342637573;0.0
2;0,0,2,2;28.657980531401954;0.0;29.300089975361765;30.612445916335
3;2,0,0,2;29.307012062855954;29.25859113811383;28.59376732648875;31.468805139022386
4;0,2,2,0;28.855932552441022;31.218279534849756;28.152230173579934;29.538881587145653
5;2,0,2,0;26.2396816447543;25.86272150450492;0.0;31.870140204435806
6;4,0,0,2;18.096422119948237;21.524150639931975;24.67407151978731;29.75600496408652
```

Figure 15 Parent Q-table representation

Child:

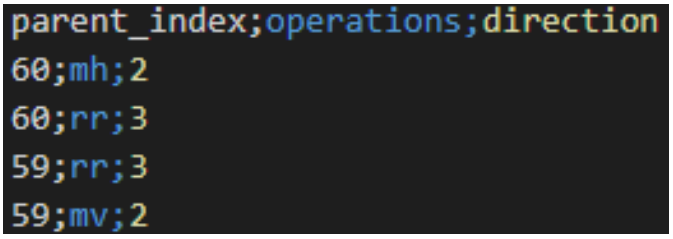

Figure 16 Child Q-table representation

G: Environment functions

Core interaction

The environment is the game logic of the game 2048. It represents an interface for the agent to interact with the game. It is implemented as a class that manages the grid, applies actions and returns rewards for each chosen action.
The environment contains the grid on which the game is played. The user can determine the grid size by using the "–grid" or "-g" flag when running the train module in the main.py script.
The Grid is initialized with zeros and two tiles are spawned at the beginning of the game followed with 1 tile after every legal action.
The main function of the environment is to give the agent the next state the reward and a terminal condition. This is done via the step() function. It receives as an argument from the agent a action a. This can be up, down, left and right.
It merges the tiles in the corresponding direction and calculates the reward based on the merged tiles. The legal actions are determined via the function find_available_actions() which checks all tiles to see if a given direction is allowed or not.
If a move was valid the step() function creates a new random tile at a random position and returns the state to the agent together with the reward gained from the chosen action.

Reward Function

The reward is based on the value of all merged tiles. If a move was deemed ineffective or bad a penalty is applied.
This reward is calculated though following function:

$$r_t = value\ of\ tiles\ merged\ in\ row\ or\ column - 1 + 0.1 * the\ number\ of\ new\ empty\ tiles \tag{48}$$

The new empty tiles are counted as a bonus as this encourages open board states which makes it easier for the parent-child-splitting to occur.

H: Agent functions

Q-table representation

The Q-table is implemented as a normal dictionary to save time. The lookup time of a python dictionary is O(1) which helps in the finding of children. Since every step requires the agent to read and write action value pairs for a given state the fast lookup is essential for the training efficiency. The states are saved as a flattened tuple and used as the key for the lookup.

$$\begin{bmatrix} 0 & 2 \\ 2 & 4 \end{bmatrix}$$

Is transformed into the tuple:

$$(0, 2, 2, 4)$$

The reason the states are saved as tuple is so they can be directly used as keys in python and tuples reduce the number of storage space needed.
The value is the list of actions for that state. If a state is encountered for the first time during the training a new directory entry will be created automatically. The action values would be initialized as 0 in this case. After every loop the action values will get updated by the learning rule as the agent trains.
In addition the agent also works with 2 additional Q-tables the parent Q-table which is a smaller version of the original Q-table and a child Q-table which only saves the connection to the parent the best direction and the operations to be applied to the parent.

Action Selection

Action selection is limited by the number of allowed actions provided by the environment. Through find_available_actions() the agent is limited to only possible actions which reduces decision wasting time and ensures compatibility between the learning process and the game.
The agent decides between exploitation and exploration by choosing a random number between 0 and 1 and then comparing it to ε. If the random number is greater or equal to ε the agent performs exploitation if not the agent performs exploration.
If the agent is in exploitation mode the agent searches through the Q-table until finding the matching states and compares their action-values. It chooses the highest value and performs the action. If the state is not yet stored it falls back to a random action-value.

This procedure can be summarized as the following:

1. with probability ε, choose a random valid action
2. with probability $1 - \varepsilon$, choose the valid action with the highest Q-value
3. if the state is unknown, choose a random valid action

4. if several valid actions share the same maximum value, choose randomly among them

The learning process of the agent is based on the standard Q-learning principle. After each loop with the environment the agent updates the Q-values of the selected action in the state. But in this prototype no explicit discount factor is used and the Q-learning update rule is therefore:

$$Q(s,a) \leftarrow (1-\alpha) * Q(s,a) + \alpha * (r + \max'_a Q(s',a')) \tag{24}$$

Training loop

The training loop defines the repeated interaction between the agent and the environment over a predefined number of episodes chosen by the user. It is used to gradually improve the agents Q-values.

At the beginning of each new episode the agent resets the environment with the environment function reset() and a new initial state is returned. The agent then interacts with the environment via actions until the episode ends. For each step the environment sends the agent all possible actions. The agent then selects an action via the previously described action selection utilizing the ε – greedy strategy. The agent interacts with the environment via the step() function in which it provides the selected action and the environment returns the next state and the reward given that action.

The agent then updates its Q-table via the previously discussed learning process. After a from the user decided amount of episodes the Q-table can be saved periodically to ensure no data loss during outages. The exploration rate ε is reduced and the next episode starts. In this thesis the Q-table is separated into parent and child Q-tables at the end of every episode but in theory it can be done at any point. To save RAM space and CPU usage the Q-table is separated into batches on which the program then runs the separation.

I: Exploration of the resulting data

Storage reduction continuation

In the three by three grid the storage reduction reaches its peak. It shows the strongest storage reduction of any grid size. At depth 1 the parent child representation already reduces storage by 28 percent. At depth 3 the reduction already reaches a maximum of 55 to 56 percent. The variation in percentage between higher depth occur due to the randomness of action selection via the agent. The reduction is storage space indicates the usefulness of the parent child abstraction in medium sized environments where the number of states is large enough to find similarities, but the state space and grid size is not too large for structural similarities to never occur.
The 4 by 4 environment shows a the same effect. The effectiveness of the abstraction strongly on the grid size and the transformation depth. At depth 1 the 4 by 4 grid reduces the required storage by 3 percent and at higher depths the storage reduction percentage again reaches the max with between 5 and 6 percent. The 3 by 3 environment benefits most from the implemented operation set. The 2 by 2 grid shows that it benefits less because the size of the Q-table is already small enough and the 4 by 4 environment benefits only slightly but takes up to much time in order for it to be deemed useful. Given the current operation set of scaling, rotating and mirroring it is therefore clear that the 3 by 3 environment shows the best results.

Total Q-error continuation

In addition to the average error the total Q-value error was also measured. The total error is the sum of all absolute Q-value differences across all compared states and actions. Unlike the average error this value is strongly influenced by the number of total states in the Q-tables.

Table 9 Total Q-value errors

Total Q-Error					
Grid Size	**Depth 1**	**Depth 3**	**Depth 5**	**Depth 7**	**Depth 10**
2x2	0	375,1574	411,7131	836,2551	816,8557
3x3	313815,7977	661935,4738	703360,8183	703820,2905	700026,4817
4x4	102717,0748	258226,9549	263227,6556	267067,6247	269681,4148

The total Q-value error increases with the number of compared states but for the 4 by 4 environment the total error was lower than the three by three environment despite the much larger

number of states. This can be explained by the lower abstraction ratio. Since fewer abstracted states result in fewer reconstructed states. Therefore a smaller amount of Q-values needed to be changed from parent to child.

J: Evaluation of the results

As already mentioned the experimental results show that the parent-child abstraction approach can reduce the storage needed to safe a Q-table by dividing it via structural similarities. However it depends strongly on the size of the states or grids in case of 2048 and the total number of states. The strongest result was found via the medium sized environment. The 3 by 3 sized environment reached an child abstraction ratio of approximately 70 percent and the storage reduction reached up to 56 percent. This reflects on more than half of the original storage requirements being reduced. It showed that similarities between states in an environment like 2048 can be identified and represented via structural operations such as rotation, mirroring and scaling applied on the matrix.

The two by two environment showed a smaller but still visible storage improvement. At higher depths this resulted in 25 percent storage reduction. At a small depth of 1 this approach however showed to use more storage due to the additional overhead of the child table. The largest environment with a grid of 4 by 4 showed the problems with the parent child abstraction. It resulted in the weakest relative storage reduction with just between 5 to 7 percent. The time required for this idea takes up to much time to be deemed useful as at a deoth of 7 the time overhead reached 35,7477, meaning the splitting and reconstruction together required more than 35 time the training time.